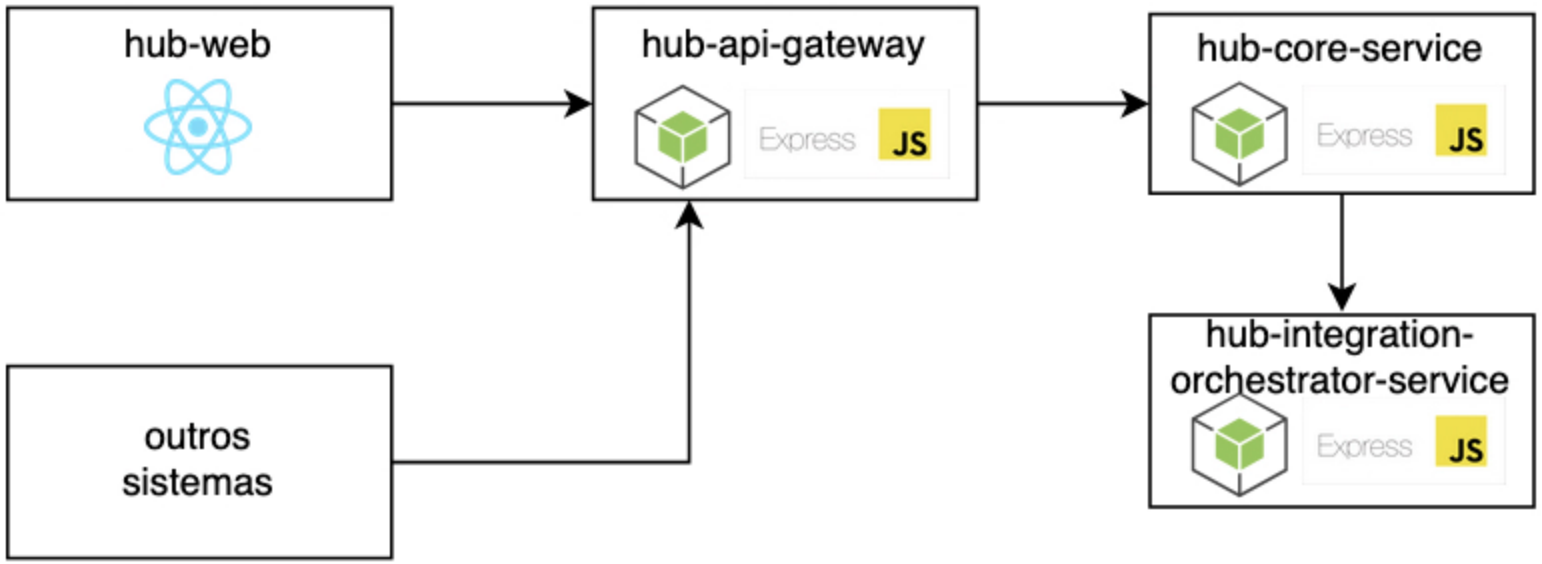
Projeto CAPEX
Hub de Obras
Arquitetura – V 1.0
Diagrama da Serviços/Componentes

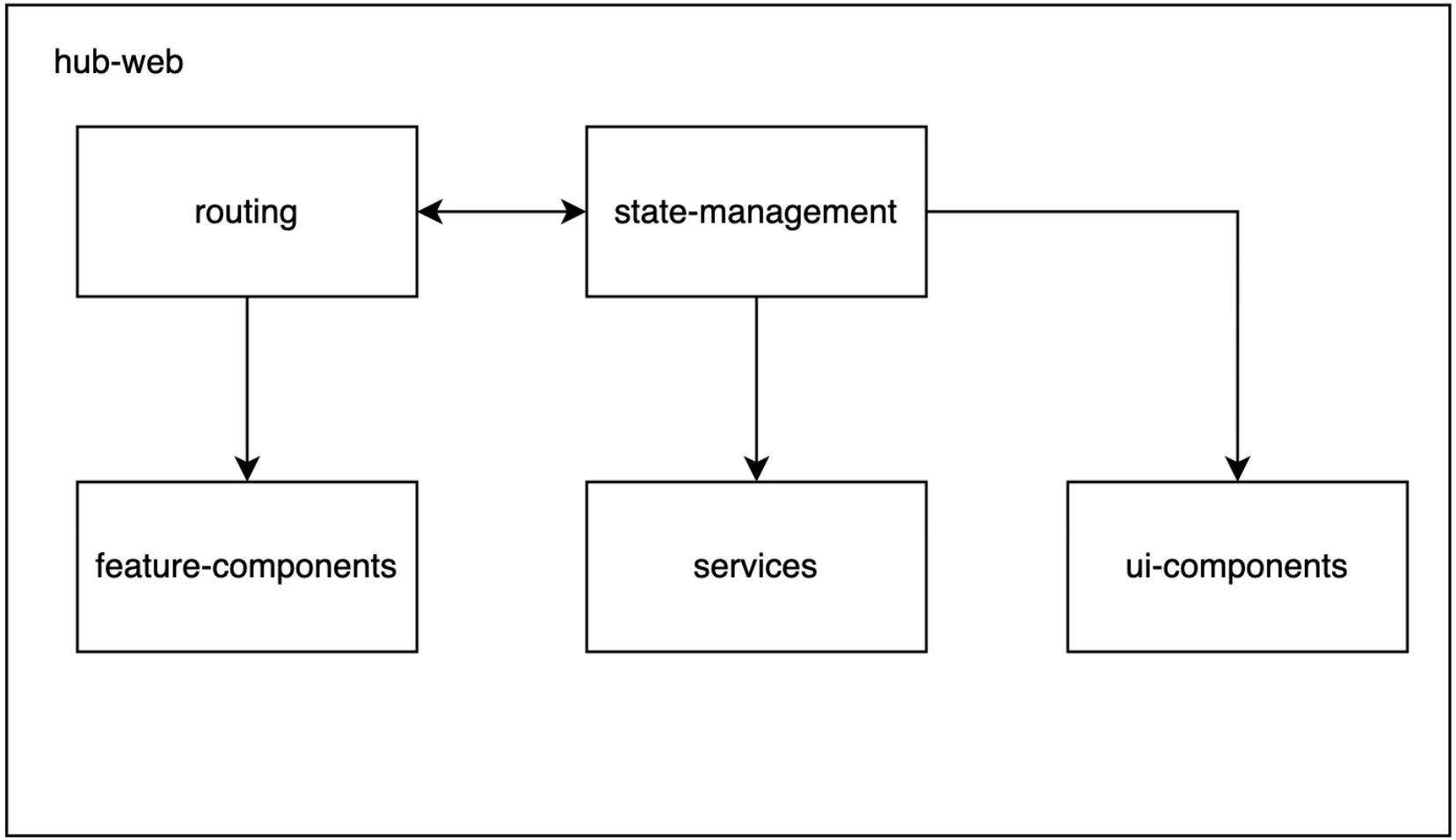
2025 Stefanini Brasil. Todos os direitos reservados.

Definição dos "Serviços" do Hub de Obras



2025 Stefanini Brasil. Todos os direitos reservados.

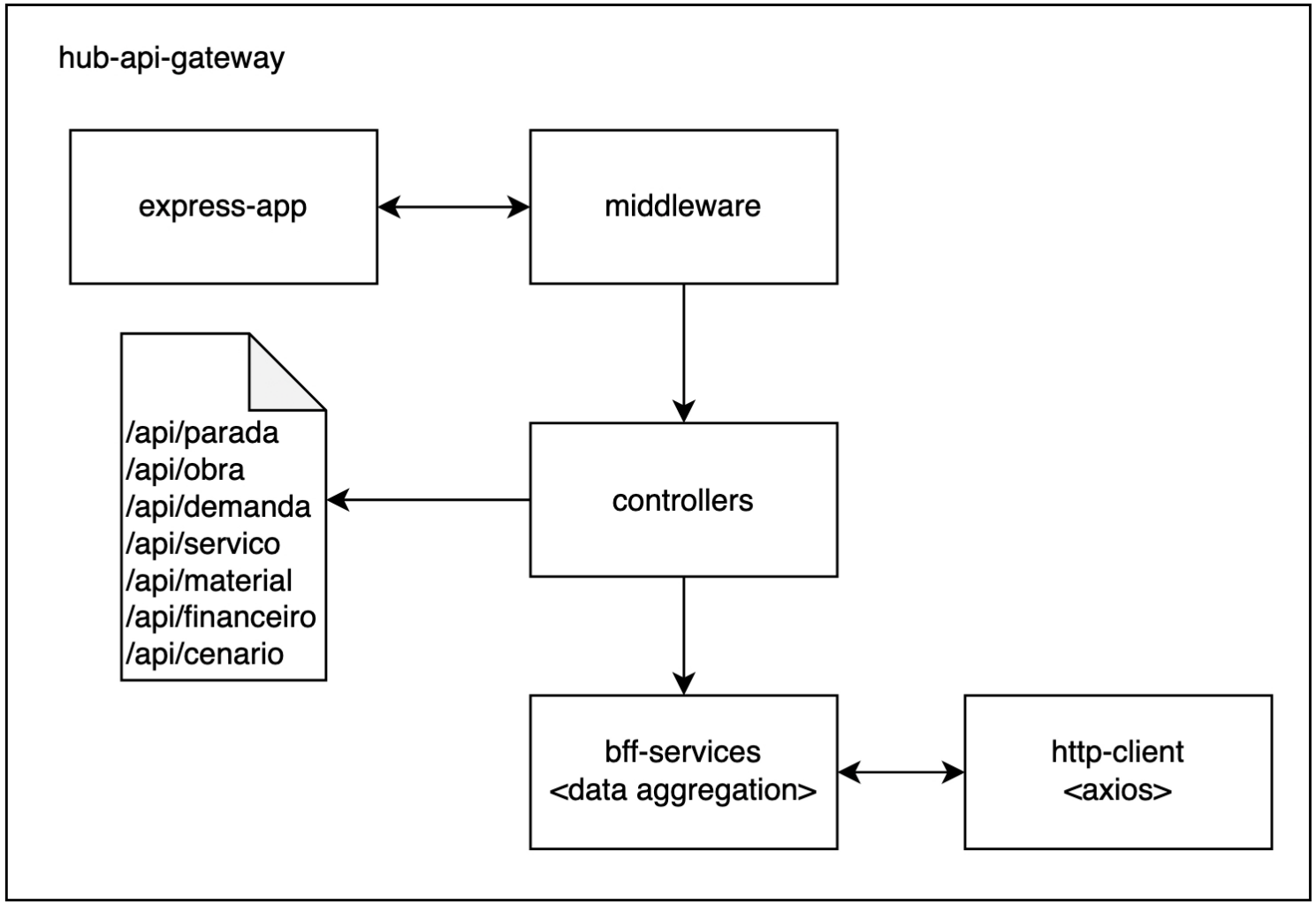
Detalhamento lógico do hub-web



Estrutura lógica inicial

2025 Stefanini Brasil. Todos os direitos reservados.

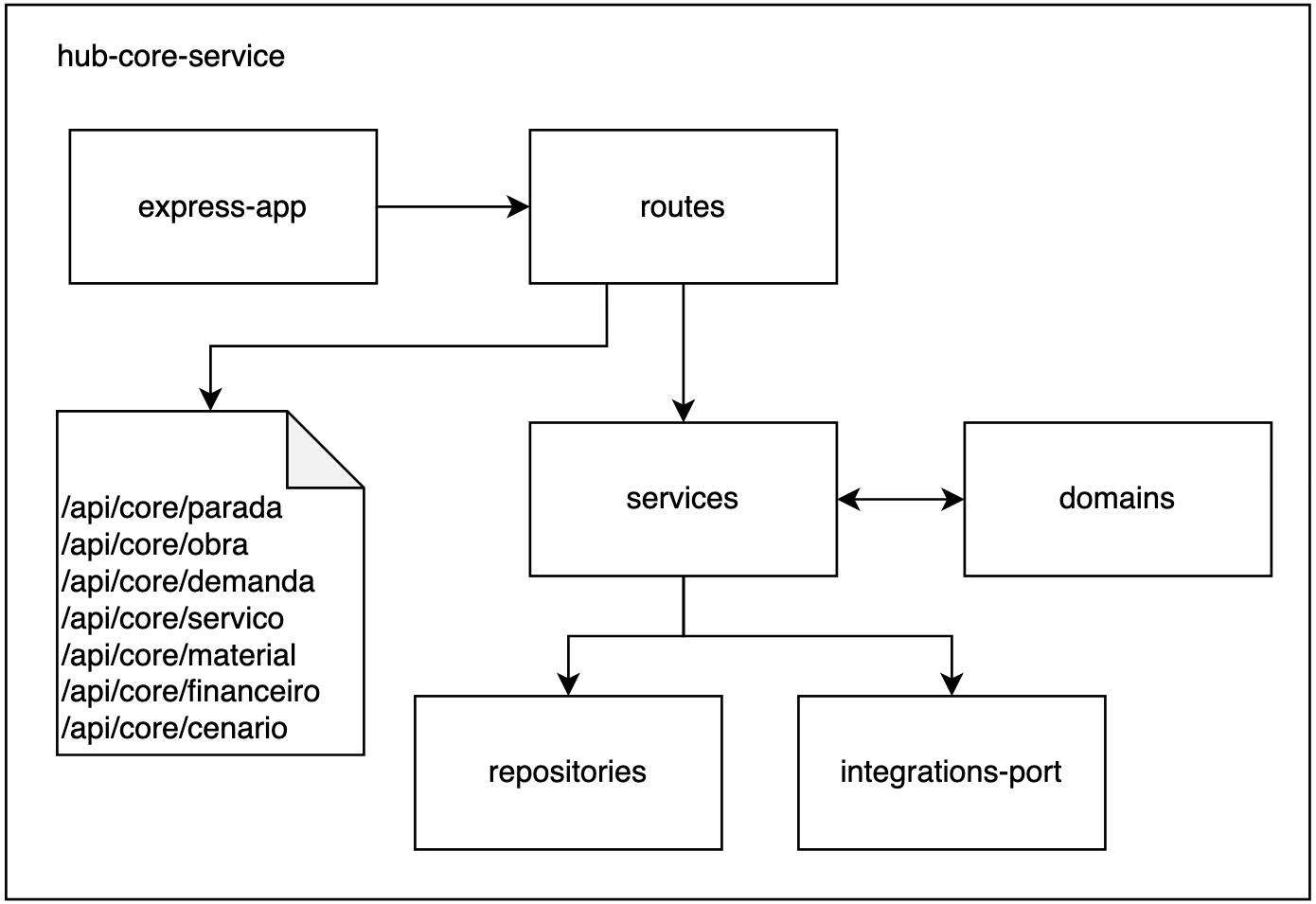
Detalhamento lógico do hub-api-gateway



Estrutura lógica inicial
Api/endpoints listadas -> sugestão inicial

2025 Stefanini Brasil. Todos os direitos reservados.

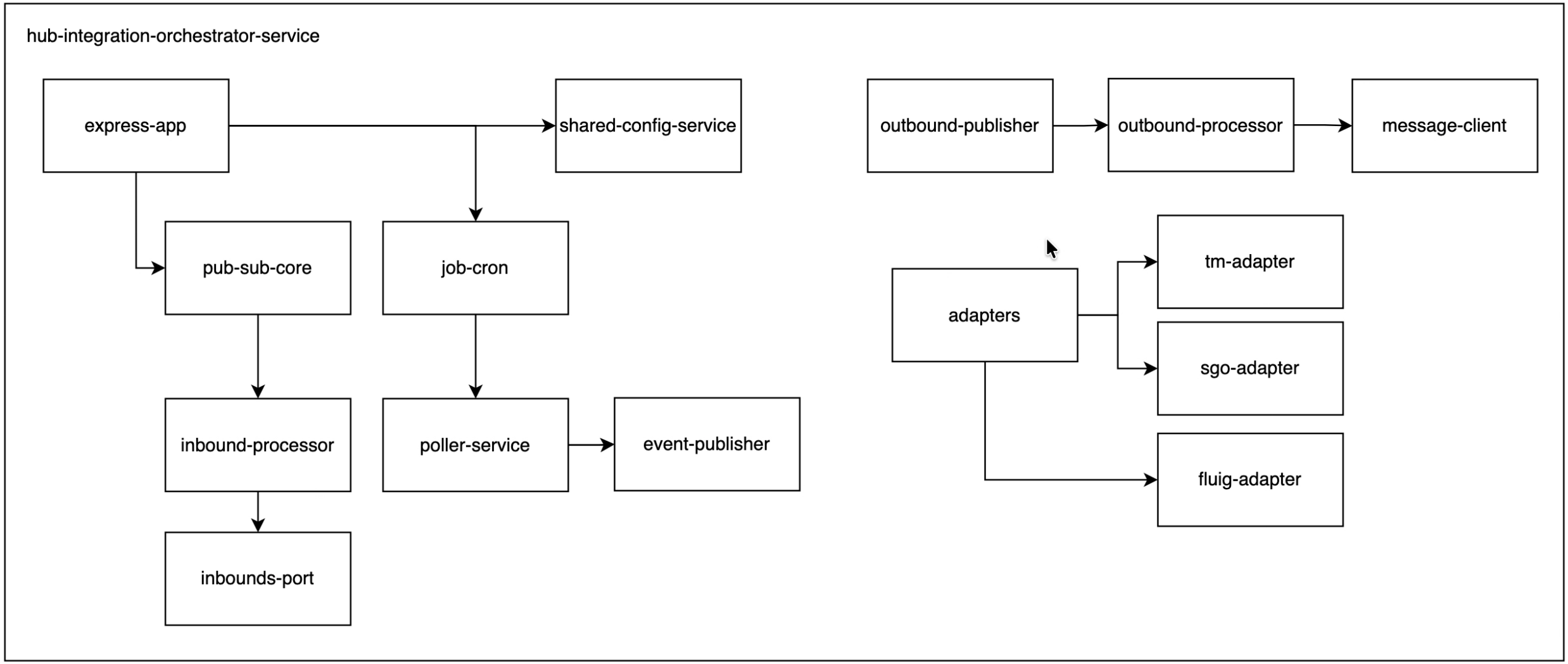
Detalhamento lógico do hub-core-service



Estrutura lógica inicial
Api/endpoints listadas -> sugestão inicial

2025 Stefanini Brasil. Todos os direitos reservados.

Detalhamento lógico do hub-integration-orquestrator-service



Estrutura lógica inicial

2025 Stefanini Brasil. Todos os direitos reservados.

Detalhes dos Serviços/Componentes

Componentes Físicos (repositórios, deployment):
- hub-web -> projeto de frontend
- hub-api-gateway -> serviço de backend para atuar como API Gateway/BFF próprio e roteamento para o hub-core-service
- hub-core-service -> serviço único responsável pelas processamento das regras de negócio do Hub de Obras
- hub-integration-orquestrator-service -> serviço responsável pelas integrações com os sistemas externos e o hub-core.

Princípios sobre separação física dos serviços **hub-core-service**:
- Considerando as informações coletadas no Discovery, definir um modelo inicial de microserviços por domínio irá gerar aumento:
    - Complexidade de deploy e observabilidade
    - Uso de mais recursos de infraestrutura -> gera mais custos
    - Complexidade de setup da estrutura inicial dos projetos de código e ambiente de desenvolvimento
- Onde os benefícios de partir para uma arquitetura orientada a microserviços não são justificáveis neste momento, assim…
- Partiremos da definição para o componente hub-core-service, com a estratégia de "monorepo", organizados logicamente por módulos/domínios de negócio, gerando um único artefato de "deploy":
    - Permite flexibilidade para crescer
    - Proteção do Core do Hub de Obras
    - Isola complexidades de integração
    - Domínios mapeados são fortemente encadeados
    - Centraliza as regras de negócio, persistência e geração de eventos de integração
- nodejs, express, sequelize
- Não deve ser pro jetado para receber/responder payloads usando classes de Model (mapeamento de entidades de banco de dados). Deverá ter DTO's para tráfego de entrada/saída, protegendo o Modelo de domínio

2025 Stefanini Brasil. Todos os direitos reservados.

Detalhes dos Serviços/Componentes

hub-web:

- projeto de código para o frontend web/pwa usando o framework React
    - App em dispositivos Android e IOS deverão ser via PWA
    - Features para o app PWA não foram identificadas necessidades de requisitos fortes de hard native (camera, gps, push sofisticado, cameras avançadas, sensores)
    - Stack baseada em JS -> mesma do backend, potencializa equipe de dev
    - Não usar react-native permite menor complexidade de ambiente de desenvolvimento, não exige tools mobile (Xcode, Android Studio)
- design das páginas web deve ser responsivo, sem adequações por tamanho de tela utilizada
- para páginas específicas do pwa, devem ser projetadas seguindo a prática "mobile first"
- realiza autenticação via MSAL library para fluxo OAUTH2/OIDC no Entra ID

hub-api-gateway (BFF):

- nodejs, express, sequelize
- Autenticação, CORS
- Endpoints orientados ao UsereXperience das funcionalidades
- Permite orquestração simples de serviços e agregação de múltiplas chamadas do hub-core
- OpenAPI/Swagger para documentação de todas APIs
- Aplicar handler para idempotency
- Valida tokens JWT no Entra ID
- Propaga via header o token na chamada dos endpoints do hub-core
- Não deverá:
    - Implementar regras de negócio
    - Conhecer detalhes de integrações com outros sistemas
    - Acesso as bases de dados (exceto Redis)
    - Integração com qualquer outro sistema que não seja o hub-core
    - Sem uso de SDKs específicos

2025 Stefanini Brasil. Todos os direitos reservados.

Detalhes dos Serviços/Componentes

hub-integration-orchestrator-service:

- Consome eventos do hub-core
- Encaminha requests para os adapters por sistema a partir de identificador (que deverá ser definido em tempo de desenvolvimento do projeto) presente nos requests http
- Módulos internos:
  - Adapters Pub/Sub:
    - Consomem (Sub) dos tópicos de Inbound do Azure Service Bus
    - Publicam (Pub) payloads (mensagens) nos tópicos de Oubound do Azure Service Bus
  - Adapter Cron:
    - Jobs schedulados para consulta de dados de Bases/Views do TM Master e SGO (ou para futuros sistemas)
- Deverão ser definidos interfaces para implementação dos adapters específicos de cada integração, onde:
  - A interface define o "contrato" das operações e dados (payload) que atenda o hub-core
  - Os adapters devem implementar as interfaces, viabilizar as integrações técnicas e retornar os payloads para o hub-core conforme a definição dos contratos (proteção, isolamento e autonomia do hub-core)
  - Deverá usar SDKs específicos para viabilizar tecnicamente as integracões
- Aplicar Idempotent Consumer para todos os handlers de fila

observabilidade:

- hub-apigateway deve gerar traceID e propagar via http header
- hub-integration-orquestrator devem carregar correlationId (messageCorrelationId)
- hub-apigateway, hub-service e hub-integration-orquestrator devem escrever logs via stdout/stderror

2025 Stefanini Brasil. Todos os direitos reservados.

Detalhes dos Serviços/Componentes

Definições gerais para todos os componentes:

- manter coesão e baixo acoplamento
- estrutura dos componentes de backend devem seguir o padrão MVC (Model View Controller):
    - domínios
    - Infra
    - agrupar código por domínio funcional, dentro subpastas com todo o MVC
- usar dependency injection entre os objetos internos de cada serviço:
    - habilita testes unitários com estratégia de mock
    - reduz acoplamento entre objetos
- implementar handlers/middlewares para tratamento de erros:
    - não expor detalhes técnicos das exceptions como resposta para os clientes (front, clientes de APIs)
    - seguir o padrão Restful na definição dos códigos de status de resposta (aplicacão padronizada dos ranges 200, 400 e 500)
- Typescript:
    - Types compartilhados entre BFF e core

- seguir o padrão Restful para formação dos endpoints:
    - uso de subtantivos (singular e plural)
    - métodos HTTP -> para definir qual operação será realizada (GET, POST, PUT, PATCH, DELETE)

2025 Stefanini Brasil. Todos os direitos reservados.